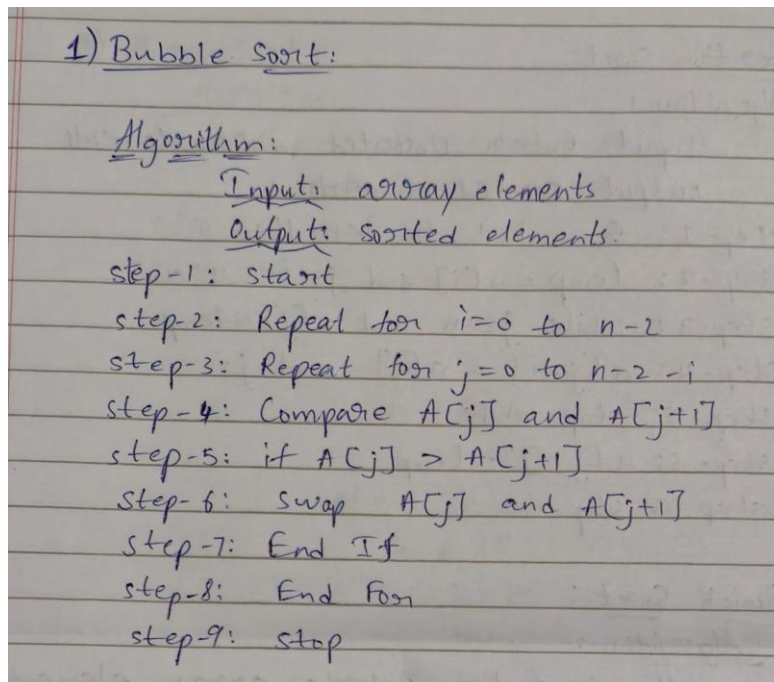


Practical – 1

Aim: To Implementation and time analysis of bubble sort algorithm.

Algorithm for Bubble sort:



Program of Bubble sort:

```
#include <iostream>
```

```
using namespace std;
```

```
class BubbleSortApp
```

```
{
```

```
public:
```

```
    int arr[100], n;
```

```
    void arrayInput()
```

```
    {
```

```
        cout << "Enter number of elements: ";
```

```
        cin >> n;
```

Name: B. Manoj Karthik

Enrollment Number:92460118094



```
cout << "Enter the elements: ";

for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

// Bubble Sort Function
void sort()
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
    }  
    }  
    }  
};
```

```
int main()  
{  
    BubbleSortApp bs;  
    bs.arrayInput();  
  
    cout << "\nArray before sorting: ";  
    bs.printArray();  
  
    bs.sort();  
  
    cout << "\nArray after Bubble Sort: ";  
    bs.printArray();  
    return 0;  
}
```

Output:

```
Enter number of elements: 5  
Enter the elements: 45 18 72 63 9  
  
Array before sorting: 45 18 72 63 9  
  
Array after Bubble Sort: 9 18 45 63 72  
  
Process returned 0 (0x0)    execution time : 40.158 s  
Press any key to continue.  
|
```

Time complexity:

Time Complexity:

For inner loop
 $i=0 ; j=0 \text{ to } n-2$
 $\rightarrow n-1 \text{ times}$
 $i=1 ; j=0 \text{ to } n-2$
 $i=2 ; j=0 \text{ to } n-3$
 So, $(n-1) + (n-2) + (n-3) + \dots + 1$
 $\Rightarrow \frac{(n-1)(n-1+1)}{2} = T(n)$
 $T(n) = \frac{(n^2 - n)}{2}$
 we ignore constant & take high expo
 $T(n) = n^2$
So, $T(n) = O(n^2)$

i) Best Case: The array is sorted
 no. of comparisons: $n-1$
 $T(n) = n-1$
 $T(n) = O(n)$

ii) Average case:
 $(n-1) + (n-2) + (n-3) + \dots + 2 + 1$
 $1 + 2 + 3 + \dots + (n-1)$
 $= \frac{n(n-1)}{2}$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

Worst case: The array is in reverse order
 $(n-1) + (n-2) + (n-3) + \dots + 2 + 1$
 $1 + 2 + 3 + \dots + (n-1)$
 $= \frac{n(n-1)}{2}$
 $T(n) = \frac{(n^2 - n)}{2}$
 $= T(n) = O(n^2)$



Aim: To Implementation and time analysis of selection sort algorithm.

Algorithm for Selection sort:

2) Selection Sort:
Algorithm:
Input: Unsorted array of elements.
Output: Sorted array
step-1: for $i=0$ to $i \leq n-1$ Then
step-2: Select $\text{min}=i$ Then do
step-3: for $j \rightarrow i+1$ to $j \leq n$ Then
step-4: if $a[j] < a[\text{min}]$ then do
step-5: $\text{min}=j$
step-6: stop if and for
step-7: if $\text{min} \neq i$ then do
step-8: Swap both
 $\text{temp} = a[i];$
 $a[i] = a[\text{min}];$
 $a[\text{min}] = \text{temp};$
step-9: stop if

Program of Selection sort:

```
#include <iostream>
```

```
using namespace std;
```

```
class SelectionSortApp
```

```
{
```

```
public:
```

```
int arr[100], n;
```

```
void arrayInput()
```

```
{
```

Name: B. Manoj Karthik

Enrollment Number:92460118094



```
cout << "Enter number of elements:";
cin >> n;
cout << "Enter the elements: ";
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}
}
```

```
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
// Selection Sort Function
void sort()
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }
    }
}
```

```
        if (minIndex != i)
        {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

};

int main()
{
    SelectionSortApp ss;
    ss.arrayInput();

    cout << "\nArray before sorting: ";
    ss.printArray();

    ss.sort();
    cout << "\nArray after Selection Sort: ";
    ss.printArray();

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter the elements: 4 3 9 6 5

Array before sorting: 4 3 9 6 5

Array after Selection Sort: 3 4 5 6 9

Process returned 0 (0x0)   execution time : 8.987 s
Press any key to continue.
|
```



Time complexity:

Time Complexity:

for () { $\rightarrow n$		$T(n) = (n+1)(n+1+1)$
min index = 1; $\rightarrow 1$		$= (n+1)(n+2)$
for () { $\rightarrow n$		$= n^2 + 2n + n + 2$
if () { $\rightarrow 1$		$T(n) = n^2$
if () { $\rightarrow 1$		$= O(n^2)$

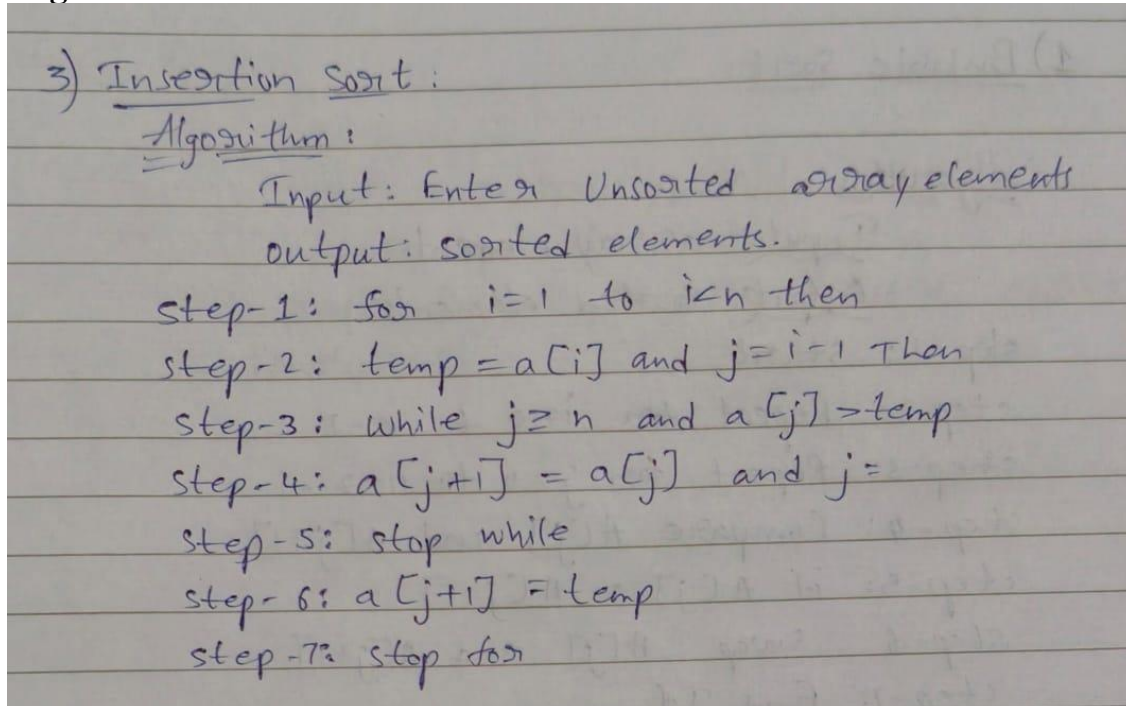
(2) i) Best case:
Perform all comparison
 $(n-1) + (n-2) + \dots + 2 + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $= \frac{(n^2 - n)}{2}$
 $T(n) = O(n^2)$

ii) Average case:
check every pass
 $T(n) = (n-1) + (n-2) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

iii) Worst case:
all comparison
 $T(n) = O(n^2)$ //

Aim: To Implementation and time analysis of Insertion sort algorithm.

Algorithm for Insertion sort:



Program of Insertion sort:

```
#include <iostream>

using namespace std;

class InsertionSortApp
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements:";
        cin >> n;

        cout << "Enter the elements:";
```



```
for (int i = 0; i < n; i++)  
{  
    cin >> arr[i];  
}  
}
```

```
void printArray()  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}
```

// InsertionSort Function

```
void sort()  
{  
    for (int i = 1; i < n; i++)  
    {  
        int key = arr[i];  
        int j = i - 1;  
  
        while (j >= 0 && arr[j] > key)  
        {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
  
        arr[j + 1] = key;  
    }  
}
```

```
    }  
}  
};  
  
int main()  
{  
    InsertionSortApp is;  
    is.arrayInput();  
  
    cout << "\nArray before sorting:";  
    is.printArray();  
  
    is.sort();  
  
    cout << "\nArray after Insertion Sort: ";  
    is.printArray();  
  
    return 0;  
}
```

Output:

```
Enter number of elements: 5  
Enter the elements: 3 1 9 5 2  
  
Array before sorting: 3 1 9 5 2  
  
Array after Insertion Sort: 1 2 3 5 9  
  
Process returned 0 (0x0)    execution time : 16.173 s  
Press any key to continue.  
|
```



Time complexity

Time Complexity:

```

for ( ) {
    key = arr[i];
    j = i - 1;
    while ( ) {
        arr[j+1] = arr[j];
        j = j - 1;
    }
    arr[j+1] = key;
}

```

$\rightarrow n$
 $\rightarrow 1$
 $\rightarrow 1$
 $\rightarrow n$
 $\rightarrow 1$

$T(n) = (n + 1 + 1)$
 $= (n + 1 + 1)$
 $= (n + 2) (n + 3)$
 $= n^2 + 3n + 2n + n$
 $T(n) = O(n^2)$

i) Best case : array is sorted one comparison

$n - 1$
 $T(n) = n - 1$
 $T(n) = O(n)$

ii) Average case :

$1 + 2 + 3 + \dots + (n - 1)$
 $T(n) = \frac{n(n-1)}{2}$
 $= \frac{(n^2 - n)}{2}$
 $T(n) = O(n^2)$

iii) Worst case :

$(n - 1) + (n - 2) + \dots + 2 + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

Aim: To Implementation and time analysis of Merge sort algorithm.

Algorithm for Merge sort:

4) Merge Sort:
Algorithm:
Input: array elements
output: Sorted array
step-1: Merge (array, low, mid, high) Then
step-2: set i = low, j = mid + 1, k = low
step-3: while i ≤ mid and j ≤ high Then
step-4: if array[i] ≤ array[j] .Then
temp[k] = array[i]
i = i + 1
else
temp[k] = array[j]
j = j + 1
end if

k = k + 1
end while
step-5: while j ≤ high then
temp[k] = array[j] ; j = j + 1 ; k = k + 1
end while
step-6: while i ≤ mid then
temp[k] = array[i]
i = i + 1
k = k + 1
end while
step-7: for i = low, k = 0 to i ≤ high Then
array[i] = temp[k]
stop for
step-8: merge sort (array, low, high) Then do
step-9: if low < high Then do
mid = (low + high) / 2
merge sort (array, low, mid)
merge sort (array, mid + 1, high)
endif merge sort (array, low, mid, high)

Program of Merge sort:

```
#include <iostream>
using namespace std;
class MergeSortApp
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements:";
        for (int i = 0; i < n; i++)
        {
```



```
        cin >> arr[i];  
    }  
}
```

```
void printArray()  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}
```

```
void merge(int low, int mid, int high)  
{  
    int temp[100];  
    int i = low;  
  
    int j = mid + 1;  
    int k = low;  
  
    while (i <= mid && j <= high)  
    {  
        if (arr[i] <= arr[j])  
        {  
            temp[k] = arr[i];  
            i++;  
        }  
        else
```



```
{
    temp[k] = arr[j];
    j++;
}
k++;
}
while (i <= mid)
{
    temp[k] = arr[i];
    i++;
    k++;
}
while (j <= high)
{
    temp[k] = arr[j];
    j++;
    k++;
}
for (int x = low; x <= high; x++)
{
    arr[x] = temp[x];
}
}
void mergeSort(int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;
        mergeSort(low, mid);
        mergeSort(mid + 1, high);
        merge(low, mid, high);
    }
}
```

// Merge Sort Function

Name: B. Manoj Karthik

Enrollment Number:92460118094

```
void sort()
{
    mergeSort(0, n - 1);
}

};

int main()
{
    MergeSortApp ms;
    ms.arrayInput();
    cout << "\nArray before sorting:";
    ms.printArray();
    ms.sort();

    cout << "\nArray after Merge Sort: ";
    ms.printArray();
    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter the elements: 2 1 9 4 3

Array before sorting: 2 1 9 4 3

Array after Merge Sort: 1 2 3 4 9

Process returned 0 (0x0)   execution time : 29.205 s
Press any key to continue.
|
```

Time complexity:

Time Complexity :

```

mergesort (low, high) {
    if (low < high) {
        mid = low + (high - low / 2); → (1)
        merge sort (low, mid);
        merge sort (mid + 1, high); } log n
        merge (low, mid, high); → n
    }
}
merge (low, mid, high) {
    while ( ) { → n
        "
        "
    }
}

```

$T(n) = (1) (\log n) (n)$
 $T(n) = n \log n$
 $T(n) = O(n \log n)$

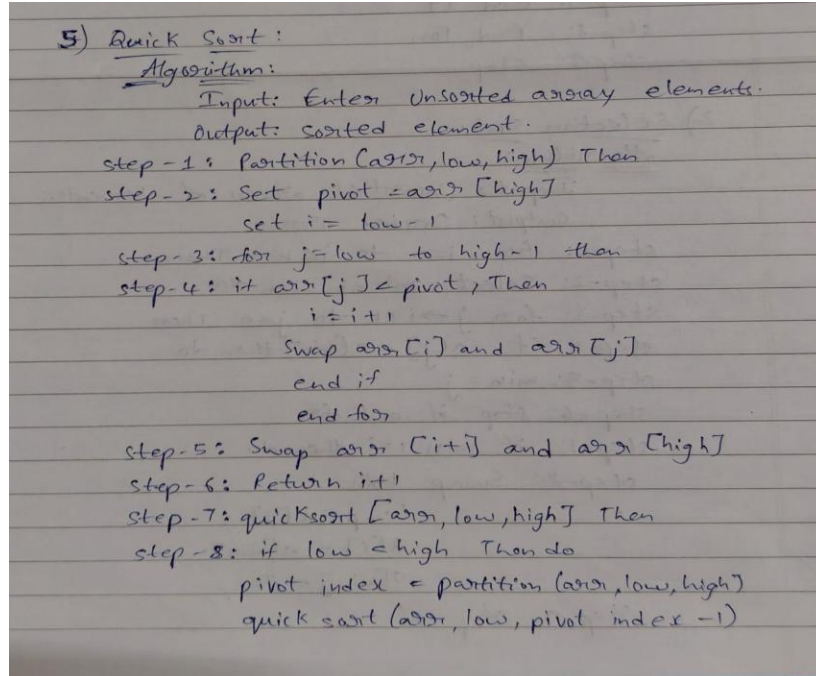
i) Best case: Divides array into two halves
 $n \rightarrow n/2 + n/2 \rightarrow n/4 + n/4 \dots + 1$
 $T(n) = n \times \log_2 n$
 $T(n) = O(n \log n) //$

ii) Average case:
 $n + n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots + 1$
 $T(n) = n \times \log n$
 $T(n) = O(n \log n) //$

iii) Worst case: Mergesort continuously divides the array into halves. Then merges all elements.
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots + 1$
 $T(n) = n + \log_2 n$
 $T(n) = O(n \log n) //$

Aim: To Implementation and time analysis of Quick sort algorithm.

Algorithm for Quick sort:



5) Quick Sort:
Algorithm:
Input: Enter unsorted array elements.
Output: sorted element.
step-1: Partition (array, low, high) Then
step-2: Set pivot = array [high]
set i = low - 1
step-3: for j = low to high - 1 then
step-4: if array [j] < pivot, Then
i = i + 1
swap array [i] and array [j]
end if
end for
step-5: Swap array [i + 1] and array [high]
step-6: Return i + 1
step-7: quickSort (array, low, high) Then
step-8: if low < high Then do
pivot index = partition (array, low, high)
quick sort (array, low, pivot index - 1)

Program of Quick sort:

```
#include <iostream>
```

```
using namespace std;
```

```
class QuickSortApp
```

```
{
```

```
public:
```

```
int arr[100], n;
```

```
void arrayInput()
```

```
{
```

```
cout << "Enter number of elements: ";
```

```
cin >> n;
```



```
cout << "Enter the elements:";

for (int i = 0; i < n; i++)

{

    cin >> arr[i];

}

}

void printArray()

{

    for (int i = 0; i < n; i++)

    {

        cout << arr[i] << " ";

    }

    cout << endl;

}

int partition(int low, int high)

{

    int pivot = arr[high];

    int i = low - 1;

    for (int j = low; j < high; j++)

    {
```



```
    if (arr[j] < pivot)
    {
        i++;

        int temp = arr[i];

        arr[i] = arr[j];

        arr[j] = temp;
    }
}

int temp = arr[i + 1];
arr[i + 1] = arr[high];
arr[high] = temp;

return i + 1;
}

void quickSort(int low, int high)
{
    if (low < high)
    {
        int pivotIndex = partition(low, high);

        quickSort(low, pivotIndex - 1);

        quickSort(pivotIndex + 1, high);
    }
}
```

```
// Quick Sort Function

void sort()

{
    quickSort(0, n - 1);
}

};

int main()

{
    QuickSortApp qs;

    qs.arrayInput();

    cout << "\nArray before sorting: ";

    qs.printArray();

    qs.sort();

    cout << "\nArray afterQuick Sort: ";

    qs.printArray();

    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter the elements: 2 1 8 4 3

Array before sorting: 2 1 8 4 3

Array after Quick Sort: 1 2 3 4 8

Process returned 0 (0x0)    execution time : 62.057 s
Press any key to continue.
|
```

